Laboratório de Organização de Computadores

Lab #3 - Alocação Dinâmica e Análise de Algoritmos em MIPS

Prof. Dr. Jean Marcos Laine
Faculdade de Computação e Informática (FCI)
Universidade Presbiteriana Mackenzie

1 Introdução e Fundamentos Teóricos

A alocação dinâmica de memória é um conceito fundamental em Ciência da Computação, permitindo a criação de estruturas de dados flexíveis que se ajustam em tempo de execução. Em MIPS Assembly, a `syscall` 9 (`sbrk`) é usada para solicitar memória na área de `heap`, o que difere da alocação estática feita em tempo de compilação. O domínio dessa técnica é crucial para projetar sistemas que gerenciam recursos de forma eficiente.

Este laboratório tem como objetivo aprofundar a compreensão sobre alocação dinâmica e a análise do desempenho de algoritmos, conectando a teoria à prática de programação em nível de máquina.

2 Orientações Iniciais

- **Ambiente de Desenvolvimento:** Utilize o simulador MARS para escrever, montar e testar seus programas. A montagem do programa pode ser feita com o atalho `F3` ou pelo menu `Run -> Assemble`.
- **Boas Práticas:**
  - Organize o código com comentários detalhados que expliquem a lógica por trás de cada bloco de instrução.
  - Use `labels` significativos.
  - Siga as convenções de registradores MIPS (ex: `$s` para variáveis de longa duração, `$a` para argumentos de funções, `$v` para valores de retorno).
- **Metodologia de Resolução:**
  - **Não use IA para gerar o código:** A IA pode ser usada para aprender, mas não para substituir seu raciocínio. Use-a para entender, revisar e melhorar o que você fez — nunca para gerar o trabalho pronto. O código e o relatório devem demonstrar sua compreensão e autoria.
  - **Rastreie a execução:** Utilize o modo passo-a-passo (`F7`) do MARS para observar o comportamento dos registradores e da memória. Esta etapa é crucial para entender como os dados se movem entre os segmentos de memória e como os ponteiros são manipulados.
  - **Justifique suas escolhas:** No relatório final, você deverá explicar por que certas instruções foram usadas, como os endereços de memória foram calculados e como a sua implementação resolveu o problema de forma eficiente.

3 Práticas de Laboratório

Exercício 1: Análise da Eficiência do Bubble Sort na Alocação Dinâmica (5,0 Pontos)

O Bubble Sort, apesar de ter uma complexidade de tempo de $O(n^2)$, é uma ótima ferramenta didática para visualizar o processo de ordenação e o custo de cada operação. Neste exercício, você irá além da simples implementação do algoritmo para analisar sua eficiência em um contexto de alocação dinâmica.

Descrição do Problema:

Implemente o algoritmo Bubble Sort em MIPS Assembly para ordenar um array de números inteiros. O programa deve medir e exibir o número total de comparações e de trocas realizadas, fornecendo uma base para a análise de desempenho.

Rubrica de Avaliação:

1. **Interface e Alocação de Memória (1,0 Pontos):**
   - **Leitura de N e Validação (0,25 pto):** O programa solicita e lê a quantidade de números (N) e valida se $N > 0$.
   - **Alocação Dinâmica na Heap (0,5 pto):** O programa calcula a quantidade correta de bytes ($N \times 4$) e aloca a memória usando `sbrk`.
   - **Leitura e Armazenamento (0,25 pto):** O programa lê os N números e os armazena corretamente no array alocado dinamicamente.
2. **Função de Ordenação e Contadores (2,5 Pontos):**
   - **Implementação do Bubble Sort (2,0 pto):** A função `bubble_sort` está corretamente implementada e ordena o array.
   - **Contagem de Comparações e Trocas (0,5 pto):** O programa incrementa corretamente os contadores de comparações e trocas dentro da função de ordenação.
3. **Saída de Resultados e Análise (1,5 Ponto):**
   - **Impressão dos Resultados (0,25 pto):** O programa exibe o array ordenado e os valores dos contadores de forma clara.
   - **Análise no Relatório (1,25 pto):** O relatório contém a análise solicitada, discutindo a relação entre as contagens e a complexidade do algoritmo, e a comparação entre alocação estática e dinâmica.

Exercício 2: Gerenciamento de Memória para um Sistema de Categorização de Dados (5,0 Pontos)

Este exercício exige que você gerencie diferentes segmentos de memória e construa uma estrutura de dados fundamental (a lista encadeada) a partir do zero, usando alocação dinâmica. A sua abordagem deve demonstrar a compreensão da interação entre o software e o hardware.

Descrição do Problema:

Desenvolva um programa em MIPS Assembly que categoriza números de um array estático em duas listas encadeadas separadas: uma para números pares e outra para números ímpares.

Rubrica de Avaliação:

1. **Configuração e Lógica de Categorização (1,5 Ponto):**
   - **Array Estático e Ponteiros (0,5 pto):** O array estático de 10 números é declarado corretamente, assim como os ponteiros `head_par` e `head_impar` (inicializados em zero).
   - **Lógica de Paridade (0,5 pto):** A lógica para determinar se um número é par ou ímpar, usando a operação `andi` com uma máscara de bits, está implementada corretamente.
   - **Controle de Fluxo (0,5 pto):** O programa percorre todos os elementos do array estático e direciona cada número para a função de inserção correta.
2. **Função de Inserção e Alocação Dinâmica (1,75 Ponto):**
   - **Alocação Dinâmica para o Nó (0,25 pto):** A função `inserir_no_final` aloca dinamicamente 8 bytes para cada nó usando `sbrk`.
   - **Manipulação de Ponteiros (1,5 pto):** A função insere o novo nó no final da lista, manipulando corretamente os ponteiros para manter a integridade da estrutura.
3. **Saída e Análise (1,75 Ponto):**
   - **Funções de Impressão (0,5 pto):** As funções `imprimir_pares` e `imprimir_impares` percorrem e exibem o conteúdo de suas respectivas listas de forma organizada.
   - **Análise no Relatório (1,25 pto):** O relatório explica as diferenças entre alocação estática e dinâmica e detalha a lógica da manipulação dos ponteiros, incluindo como a solução lida com listas vazias.

4 Reduções de Nota

A nota da atividade é calculada com base na pontuação detalhada acima para cada exercício e com reduções de nota para os seguintes itens indesejáveis:

ITEM INDESEJÁVEL	REDUÇÃO DE NOTA
O código não compila ou não atende aos requisitos do enunciado	ZERO
Arquivos fontes (`.asm`) não foram enviados	ZERO
O laboratório não foi apresentado pelo grupo	ZERO
O grupo não enviou o relatório	-5 ptos
O relatório não demonstra todos os casos de teste no exercício	-0,5 pto/exercício
Não há identificação dos integrantes do grupo nos códigos-fontes	-0,3 pto/exercício
O código não contém comentários explicando as etapas essenciais	-0,5 pto/exercício

5 O que entregar?

1. O relatório solicitado para a atividade, no formato PDF, usando o template disponibilizado.
2. Os códigos fonte em MIPS (`.asm`), nomeados de acordo com o exercício (ex: `Lab3ex1.asm`, `Lab3ex2.asm`).

Certifique-se de que o programa esteja correto, bem organizado, comentado e legível.

6 Regulamento de Apresentação e Entrega dos Projetos Práticos

6.1 Apresentação e Defesa

- A apresentação do projeto é obrigatória para a atribuição de nota. Todos os integrantes devem estar presentes para a defesa do trabalho.
- Cada aluno deve demonstrar domínio sobre a implementação, sendo capaz de explicar a lógica, justificar as decisões tomadas e realizar alterações ou correções solicitadas pelo professor no momento da apresentação, se necessário.
- A ausência na apresentação ou o desconhecimento do funcionamento do código por qualquer integrante resultará em nota zero no projeto para o mesmo.

6.2 Originalidade e Autoria

- O trabalho deve ser integralmente desenvolvido pelo grupo.
- É estritamente proibida a utilização de códigos prontos, trechos de terceiros ou gerados por ferramentas de Inteligência Artificial sem o devido entendimento.
- Casos de plágio ou compartilhamento indevido de código resultarão em nota zero e poderão levar a sanções disciplinares pela coordenação.

6.3 Entrega e Prazos

- O projeto deve ser entregue exclusivamente na plataforma especificada (ex: Moodle). Envios por outros meios, como e-mail, não serão considerados válidos e a entrega não será aceita.
- O prazo de entrega deve ser rigorosamente respeitado. Entregas fora do prazo não serão aceitas.

6.4 Organização dos Grupos

- Esta atividade deve ser realizada em grupos de **2 (dois) a 4 (quatro) alunos**. Casos excepcionais devem ser discutidos diretamente com o professor. Não é permitido trabalhos individuais.

Referências Bibliográficas

Hennessy, J. L.; Patterson, D. A.; Larus, J. R.; Machado Filho, N. *Organização e projeto de computadores: a interface hardware/software*. 5. ed. Rio de Janeiro: Elsevier, 2014.